

Neural Surrogates for Remaining Useful Life Estimation and Fault Detection on Turbofan Sensor Data

Yaswanth Deevi

Abstract

This report studies whether a learned neural surrogate can replace a classic, computationally heavier prognostics method for estimating the Remaining Useful Life (RUL) of engines from multivariate sensor time series, and whether its output supports reliable fault detection. On the NASA C-MAPSS FD001 dataset, an MLP and a BiLSTM reach a test RMSE of about 14.3 cycles over five seeds, only slightly better than a health-index similarity baseline (15.0), but they are about 35 times faster per prediction. For fault detection, a stacked combination of a surrogate threshold, an Isolation Forest and a sensor deviation rule gives the best F1 (0.80 over all test windows). A multi-seed ablation confirms the value of RUL clipping and a 30-cycle window, and shows that the risk of data leakage depends on model capacity.

1 Introduction

Estimating the health state of an engine from its sensor streams is central to predictive maintenance. Exact or physics-based estimators are often expensive at inference time, because they require per-unit fitting, iterative solvers or simulation. A *surrogate model* learns the mapping once and then answers with a single cheap forward pass, amortizing the cost of training over every later prediction. This project asks two questions:

1. How does a neural surrogate compare with classic prognostics baselines in both accuracy and computational cost?
2. Can the surrogate’s output drive a reliable fault alarm, and how should it be combined with unsupervised detectors?

2 Data

The NASA C-MAPSS turbofan degradation dataset [1] simulates engines run until failure. The FD001 subset has one operating condition and one fault mode (high-pressure compressor degradation). Each row is one operating cycle with 3 operating settings and 21 sensors. The training set contains 100 run-to-failure engines (20,631 cycles, lifetimes of 128 to 362 cycles); the test set contains 100 engines truncated before failure, with the true RUL at truncation provided (7 to 145 cycles).

For a training engine with final cycle T , the label at cycle t is $\text{RUL}_t = T - t$. For a test engine observed up to cycle T' with true remaining life R at truncation, $\text{RUL}_t = (T' - t) + R$. A window is labelled *faulty* when $\text{RUL} \leq 20$.

3 Method

3.1 Preprocessing

- **Feature selection:** features with zero variance in the training set are removed, leaving 17 features (15 sensors, 2 operating settings).
- **Normalization:** z-scores computed from training statistics only.

- **RUL clipping:** labels are capped at 125 cycles, giving a piecewise-linear target, since early wear is not visible in the sensors.
- **Windowing:** sliding windows of 30 cycles with stride 1, built per engine; the target is the RUL at the last cycle of the window.
- **Split:** 80 training engines (14,042 windows) and 20 validation engines (3,689 windows), split by whole engine so that overlapping windows of one engine never appear in both sets.

Test performance is reported on the last window of each test engine (100 samples, the standard C-MAPSS protocol) and, for detection, also on every test window (10,196 windows, 123 of them faulty).

3.2 Baselines

C-MAPSS provides no physics model, so the baselines are classic data-driven prognostics methods built on a learned *health index* (HI). The HI is a linear combination of the 15 sensors, $HI = \mathbf{w}^\top \mathbf{x} + b$, fit by least squares so that $HI \approx 1$ during each training engine’s first 30 cycles and $HI \approx 0$ during its last 30. It is smoothed with a trailing 5-cycle moving average.

- **HI trend:** fit a line to the test engine’s last 50 HI values and extrapolate to the population failure level (the mean HI at failure across training engines).
- **HI similarity** [2]: slide the test engine’s last 100 HI values along each of the 100 training trajectories, take the best-matching position in each, and predict the median remaining life of the 10 closest matches. It requires no training, but every prediction scans the whole reference library.
- **Mean predictor:** always predicts the mean training RUL.

3.3 Surrogate models

- **MLP:** the 30×17 window is flattened to 510 inputs and passed through two ReLU layers (128 and 64 units, dropout 0.2) to a single output. It has 73 729 parameters.
- **BiLSTM** [4]: a bidirectional LSTM with 64 units per direction reads the window cycle by cycle; the concatenated output at the last cycle passes through a small head ($128 \rightarrow 32 \rightarrow 1$). It has 46 657 parameters.

Both are trained with mean squared error, Adam [5] (learning rate 10^{-3}), batch size 256, up to 40 epochs, and early stopping with patience 8 on the validation engines. All random seeds are fixed and cuDNN is set to deterministic mode. Models are trained on a GPU; latency is measured on the CPU at batch size 1 (median of 200 calls after warm-up), so that it is comparable with the CPU-only baselines.

3.4 Fault detection

Each detector produces a score (higher means more likely faulty), and its alarm threshold is chosen to maximize F1 on the validation engines, never on the test set.

- **Surrogate threshold:** score = $-\hat{y}$, the negated predicted RUL. The tuned threshold for the MLP was $\hat{y} \leq 19.4$.
- **Isolation Forest** [3]: fit on early-life windows only (RUL at the 125 plateau), using per-sensor summary features: the mean of the last 5 cycles (level) and the least-squares slope over the window (trend), 34 features in total.
- **Rule-based:** the number of sensors whose level lies more than 3σ from the early-life mean, an interpretable analogue of co-occurring deviation rules.
- **Combiners:** OR, AND, a 2-of-3 vote, and a stacked logistic regression on the three standardized scores, fit and threshold-tuned on the validation windows.

3.5 Metrics

Regression is evaluated with RMSE, $\text{NMSE} = \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$ (equal to $1 - R^2$), and the PHM08 score $S = \sum_i s_i$ with

$$s_i = \begin{cases} e^{-d_i/13} - 1, & d_i < 0 \\ e^{d_i/10} - 1, & d_i \geq 0 \end{cases} \quad d_i = \hat{y}_i - y_i,$$

which penalizes late predictions more heavily than early ones. Detection is evaluated with precision, recall and F1, with 95% bootstrap confidence intervals over test engines. Multi-seed results are reported as mean $\pm$ standard deviation over 5 seeds; the seed changes the engine split, the weight initialization and the batch order.

4 Results

4.1 Surrogates versus baselines

Approach	RMSE	NMSE	PHM08	F1	Latency (μ s)	Params
Mean predictor	40.48	1.020	17604	0.000	≈ 0	1
HI trend	37.19	0.861	18837	0.400	297	16
HI similarity	15.02	0.140	428	0.828	2633	16
MLP	14.25 ± 0.42	0.127 ± 0.008	328 ± 34	0.802 ± 0.028	75 ± 41	73 729
BiLSTM	14.34 ± 0.59	0.128 ± 0.011	375 ± 68	0.823 ± 0.038	370 ± 192	46 657

Table 1: Accuracy and cost on the 100 FD001 test engines. F1 uses the alarm rule $\hat{y} \leq 20$. Latency is per prediction at batch size 1 on the CPU.

Table 1 shows three findings. First, the surrogates’ advantage is mainly in cost: the similarity baseline is within about 0.8 RMSE of both networks (its bootstrap 95% interval is 12.5 to 17.4), yet the MLP answers about 35 times faster, and its cost does not grow with the size of the reference library. Second, the MLP and BiLSTM are tied within seed variance. A single seed had suggested the BiLSTM was better (13.60 versus 14.27), which five seeds showed to be noise. The BiLSTM is smaller but about 5 times slower, because it must process the 30 cycles sequentially. Third, the MLP has the lowest PHM08 score, meaning its errors are the least likely to be dangerously late. HI trend extrapolation performs poorly because degradation is flat early in life and accelerates late, which a local straight line cannot capture.

4.2 Preprocessing ablation

Variant (MLP)	RMSE (clipped labels)	RMSE (true RUL)	PHM08
Window 1	17.69 ± 0.20	18.74 ± 0.17	1059 ± 109
Window 10	18.27 ± 0.30	19.18 ± 0.24	1126 ± 175
Window 30 (default)	14.25 ± 0.42	15.36 ± 0.52	328 ± 34
Window 50	16.63 ± 0.60	17.47 ± 0.71	1239 ± 1190
No RUL clipping	24.96 ± 0.99	24.73 ± 1.01	26866 ± 18869

Table 2: One change from the default per row, 5 seeds.

RUL clipping is clearly justified: removing it raises RMSE from 14.3 to 25.0, and the model remains worse even when scored against the true, unclipped RUL (Table 2). A 30-cycle window is best among those tested, but the trend is not monotonic. With a 50-cycle window, the shortest test engines (31 cycles) are mostly padding, and a few large late errors inflate the PHM08 score on some seeds. A likely confound is the fixed 40-epoch budget: the default MLP’s validation loss was still decreasing at the final epoch, and larger inputs converge more slowly.

4.3 Leakage ablation

Train/validation split	Validation RMSE	Test RMSE
By engine (default)	14.65 ± 0.64	14.25 ± 0.42
By window (leaky)	15.13 ± 0.15	14.20 ± 0.48

Table 3: Effect of splitting overlapping windows across train and validation.

Splitting by window, which places near-duplicate windows of the same engine in both sets, did not make validation performance optimistic (Table 3). Leakage only helps a model that can memorize those near-duplicates; this MLP is regularized with dropout and still underfits (its training loss stays above its validation loss), so it cannot exploit the leak. The engine-level split is kept, since a higher-capacity model would.

4.4 Fault detection

Detector	MLP surrogate		BiLSTM surrogate
	F1 last window [95% CI]	F1 all windows	F1 last / all
Surrogate threshold	0.812 [0.64, 0.94]	0.727	0.909 / 0.742
Isolation Forest	0.733 [0.52, 0.88]	0.640	0.733 / 0.640
Rule-based	0.839 [0.67, 0.96]	0.651	0.839 / 0.651
Surrogate OR rules	0.857 [0.70, 0.96]	0.665	0.909 / 0.682
Surrogate AND rules	0.786 [0.60, 0.93]	0.718	0.839 / 0.707
2-of-3 vote	0.759 [0.56, 0.91]	0.781	0.839 / 0.758
Stacked logistic regression	0.867 [0.71, 0.97]	0.803	0.786 / 0.789

Table 4: Detection F1 for seed 0. Last window: 100 engines, 16 faulty. All windows: 10196 windows, 123 faulty. Isolation Forest and the rules do not use the surrogate, so their scores are identical in both columns.

On the larger all-windows evaluation, the stacked combiner is the best detector (F1 0.803 with the MLP), ahead of voting and every single detector (Table 4). The unsupervised detectors perform worst there. They are fit on new engines, so they flag anything that differs from new, including engines that are degraded but not yet near failure; only the supervised surrogate knows where the fault boundary lies. On the last window alone, detectors cannot be ranked: with 16 faulty engines, every confidence interval spans roughly ± 0.15 and they all overlap. The rankings on the two evaluations also disagree, which confirms that last-window scores are too noisy to use on their own. Detection quality is similar whether the MLP or the BiLSTM provides the surrogate score.

5 Limitations

- Results come from one simulated subset with a single operating condition; real engines add noise, drift and varying operating points.
- No physics model is available, so the surrogate learns from RUL labels rather than imitating an expensive simulator.
- The 40-epoch budget did not let the MLP fully converge, which may affect the window-size comparison.
- Detection results are from a single seed, and the stacked combiner is fit and tuned on the same validation windows.
- Latency uses Python-level timing, so absolute values depend on the hardware; the relative ordering is stable.

6 Conclusion

A small neural surrogate matches the accuracy of a strong classic prognostics method at about one thirty-fifth of its inference cost, which quantifies the accuracy versus efficiency trade-off that motivates surrogate modelling. Its predicted RUL is a useful fault signal on its own, and a learned combination with unsupervised detectors improves detection further. The ablations support the preprocessing choices and show that the value of each design decision should be measured over several seeds, since single runs can be misleading. Natural next steps are a longer training budget, multi-seed detection results, uncertainty estimates for the RUL, and the multi-condition FD002 and FD004 subsets.

References

- [1] A. Saxena, K. Goebel, D. Simon and N. Eklund, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008.
- [2] T. Wang, J. Yu, D. Siegel and J. Lee, “A similarity-based prognostics approach for remaining useful life estimation of engineered systems,” in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008.
- [3] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008.
- [4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735 to 1780, 1997.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.